

Ali Ezzat, PhD

Principal Data Scientist — Deep Learning, Real-Time ML Systems, Optimisation & Recommender Systems

aliezzat1985@gmail.com · linkedin.com/in/aliezzat1985 · github.com/alizat

PROJECT PORTFOLIO

Twenty-one production machine learning and analytics engagements across financial services, industrial manufacturing, supply chain, healthcare and mobility. Each is presented as a STAR case study — the situation and task that defined it, the solution I built, and the outcome it delivered.

Contents

CREDIT RISK & LENDING

#	Project	Headline result
1	Credit Risk ML Platform - Multiple Financial Institutions	\$150M+ lending, 40% lower defaults, 12 institutions
2	RayaTrade - Collections Prioritisation Model	Ranked call list across ~45K borrowers/month
3	Tamweely - Credit Scoring Model and Deployment API	Deployed scoring API + production validation guide
4	B.TECH - Credit-Scoring Modelling and Production Scorer	Multi-stage scorer, 74% avg precision
5	EGBank, Zeyada and Sarwa - Additional Engagements	Three further institutions onboarded

GENERATIVE AI & LLM SYSTEMS

#	Project	Headline result
6	Raya - Alternative Credit Signals from SMS	Multilingual GPT-4o pipeline + QLoRA-distilled Llama 3.1 8B
7	Forsa - LLM-Powered Customer Engagement Assistants	GPT-4o WhatsApp assistants for activation and collections

INDUSTRIAL & SUPPLY CHAIN OPTIMISATION

#	Project	Headline result
8	Unilever - Real-Time Detergent Manufacturing Recommender	23% lower gas consumption, zero batch failures
9	Unilever - Multi-Variant Manufacturing Scheduler	Time-minimising schedules from a demand sheet
10	Procter & Gamble - Supply Network Design Optimisation	18% lower total supply-network cost
11	Grinta - Medicine Inventory Replenishment Forecasting	Cut carrying fees and stock-outs simultaneously

FORECASTING, PRICING & CUSTOMER ANALYTICS

#	Project	Headline result
12	Western Union (IBAG) - Branch Cash-Withdrawal Forecasting	Over-supply error 300% → 20%
13	Wayak Health - Patient Lead Scoring and KPI Analytics	Targeting precision 3% → 25%
14	AMAN Financial Services - Merchant Churn Prediction	79% precision on 100GB+ POS data
15	London Cab - Driver Positioning and Pricing Simulation	13% faster passenger pickup
16	Naqla - Truck-Trip Price Analytics and Estimation	Automated trip-price estimation + driver analytics
17	Sa3ar - Used-Car Fair-Price Estimation	Market dataset scraped and priced from scratch
18	Toyota - Customer Data Quality and Deduplication	Multi-identifier entity resolution

RESEARCH & OPEN SOURCE

#	Project	Headline result
19	dbparser - Open-Source Drug-Database Integration Package	50K+ downloads, active community adoption
20	PhD Research - Drug-Target Interaction Prediction	4 papers + 1 book chapter, InCoB 2016 Best Paper

EARLIER ENGINEERING

#	Project	Headline result
21	Earlier Software Projects - BRS and OTMS	3 years enterprise Java delivery, on-site deployment

CREDIT RISK & LENDING

Credit Risk ML Platform - Multiple Financial Institutions (including RayaTrade)

CONTEXT

Situation. Twelve financial institutions across Egypt and the region were making lending decisions without reliable, scalable credit assessment. Each one carried default rates it could not explain and could not reduce, and each was reviewing applications manually or through rules that no one had validated against outcomes.

Task. Architect and deploy credit-scoring pipelines for all twelve. Each pipeline had to do more than predict risk: it had to return an accept/reject decision *and* a defensible loan amount, using the client's own income-estimation formulas, debt-burden rules and lending limits.

Challenge. A model alone was never the deliverable. Credit-bureau data, eligibility criteria, policy rules and loan-limit arithmetic all had to be operationalised consistently and exposed to the client's systems as something their engineers could call in production. Every client had different policy, different data, and different tolerance for risk.

SOLUTION

Action. I built GOOD/BAD credit-scoring engines paired with client-specific policy engines. The models ingest features parsed from applicant credit-score reports and return an accept/reject decision alongside a recommended loan amount. I owned the entire lifecycle — data wrangling, EDA, feature engineering, model training, Docker containerisation, API development, deployment, production support, and direct intervention when incidents hit production.

For RayaTrade specifically, the system includes an iScore report parser, an association-analysis workflow that surfaces feature-value patterns correlated with good and bad credit behaviour, trained model and PCA artifacts, a simulation harness, and maintained policy lists covering geographic areas, professions and industry sectors. I shipped it as a containerised API with an OpenAPI specification, sample request/response payloads, local and deployed simulation scripts, and full Konan MLOps deployment support.

Rationale. Predictive signal on its own is not an operational decision. Fusing the model output with explicit, client-owned credit policy is what makes the answer usable by a loan officer and auditable by a risk committee. The simulation and API test layers exist so that every release can be validated against realistic traffic before it touches a live applicant.

OUTCOMES

Result. Across the twelve deployments the platform processes **100K+ loan applications per month** and has facilitated **\$150M+ in lending**. Average default rates fell **40%** against clients' pre-engagement baselines.

I also served as applied ML lead for Konan, Synapse Analytics' internal MLOps platform, translating what I learned running these pipelines into platform requirements — Konan now supports roughly 25 production models across about 15 client deployments.

RayaTrade - Collections Prioritisation Model

CONTEXT

Situation. RayaTrade needed to chase roughly **45,000 borrowers a month** about their upcoming instalment. Calling all of them was not viable — the collections team had a fixed number of agents and a fixed number of hours, and was spending them uniformly across customers who were overwhelmingly going to pay on time anyway.

Task. At the start of each month, tell operations exactly which funded customers to call, ranked by how much the call was likely to matter.

Challenge. The relevant failure is an instalment delayed by more than 30 days. Predicting that is only half the problem; the output had to arrive as something a collections supervisor could act on Monday morning without interpretation.

SOLUTION

Action. I built a model trained on instalment-payment history, with features drawn primarily from each customer's behaviour on preceding instalments. It predicts good/bad behaviour on the current instalment and emits a call list ordered by the likelihood that the customer will miss the payment date. The pipeline covers dataset construction, feature utilities and monthly call-list generation end to end.

Rationale. Risk ranking converts a scarce resource — human contact time — into a prioritised queue, and the handoff is deliberately simple: one ordered list per month, no dashboard to interpret, no model output for a non-technical user to second-guess.

OUTCOMES

Result. The model turns an undifferentiated 45,000-borrower calling problem into a ranked worklist, allowing collections effort to concentrate on the customers most likely to fall past 30 days.

Tamweely - Credit Scoring Model and Deployment API

CONTEXT

Situation. Tamweely needed credit scoring built entirely from its own database — no parsed iScore reports, no external bureau features to lean on.

Task. Predict GOOD or BAD repayment behaviour from a customer profile, where BAD means any payment delayed by more than 15 days. Deploy it as an API and validate it against live production data.

Challenge. A 2025 database migration changed the operating environment underneath the model. Validation was therefore not a formality: it required agreement with the client on label definitions, verification of request-schema integrity, sanity checks on response distributions, and enough matured applications to actually evaluate outcomes.

SOLUTION

Action. I built the full R workflow — data wrangling, feature engineering, EDA, model training and deployment testing — and packaged it as a Dockerised deployment bundle with an API server, documented endpoints, an OpenAPI specification, the trained model artifact, local and deployed test suites, and a documented Konan release process.

I also wrote the operating guide the client runs against production: checks for missing and wrongly typed fields, detection of suspiciously constant fields, monitoring of GOOD/BAD response ratios, and reconciliation against a client-supplied labelled test set.

Rationale. Most credit models fail in production not because the model is wrong but because the data arriving at the endpoint quietly stops matching what the model was trained on. Explicit, written validation procedures catch schema and distribution regressions early — and settling the definition of "bad repayment" in advance prevents the disagreement that otherwise surfaces months later when the numbers are disputed.

OUTCOMES

Result. Delivered a deployed, documented scoring API with an operating guide the client can run independently. Post-deployment analysis established that 34% of accepted applications in 2025 were BAD — a baseline that quantifies the size of the problem the model exists to address.

B.TECH - Credit-Scoring Modelling and Production Scorer

CONTEXT

Situation. B.TECH extends consumer financing at retail point of sale, where a credit decision has to be made in the time a customer is standing at a counter — and where the information available at that moment varies.

Task. Train, test, simulate and package credit-risk models for operational scoring.

Challenge. One model could not serve every decision point. The system needed distinct accept/reject and good/bad models, each with variants that use or exclude iScore bureau information, reflecting the reality that bureau data is not always available at the moment of decision.

SOLUTION

Action. I implemented an R pipeline covering cleaning, labelling, model development, targeted testing, production simulation, production training and scorer preparation. The delivered artifacts include trained RDS models for both accept/reject and good/bad variants, a production scorer, a test suite and Docker build configuration.

Rationale. Keeping training, testing, simulation and scoring as separate, versioned artifacts is what makes a credit model reproducible a year later — when a regulator, a client, or a successor asks why a specific applicant was rejected.

OUTCOMES

Result. A production-packaged scorer covering multiple decision stages and information-availability scenarios, within a credit-scoring portfolio averaging **74% precision** on credit-worthiness prediction.

EGBank, Zeyada and Sarwa - Additional Credit-Scoring Engagements

CONTEXT

Situation. Alongside Tamweely and B.TECH, I delivered credit-scoring work for EGBank, Zeyada and Sarwa — three further financial institutions within the twelve-client credit-risk programme, each with its own data, policy and risk appetite.

Task. Develop credit-worthiness assessment and loan-amount decisioning models, and maintain, experiment with and test the credit-risk policy workflows around them.

Challenge. Each client's lending policy differed, so the modelling approach had to transfer while the policy layer was rebuilt per institution.

SOLUTION

Action. I deployed ML models that assess an applicant's credit-worthiness and determine the loan amount to grant, supported by association analysis identifying sets of feature-value pairs that correlate with good and bad credit behaviour, and facilitated the maintenance, experimentation and testing of each client's credit-risk policy workflows.

Rationale. Association analysis complements the model by producing something a credit committee can read: explicit, human-legible patterns tied to good and bad outcomes, which supports policy discussion in a way a model score alone does not.

OUTCOMES

Result. Three further institutions brought onto the credit-scoring platform, within a portfolio averaging **74% precision** on credit-worthiness prediction.

GENERATIVE AI & LLM SYSTEMS

Raya - Alternative Credit Signals from SMS

CONTEXT

Situation. Thin-file applicants — people with little or no credit-bureau history — are effectively invisible to conventional credit scoring. They are also a large share of the addressable market. Their phones, however, hold a detailed financial record: transaction alerts, salary notifications, bill reminders, card activity.

Task. Build an embeddable service that converts raw SMS inboxes into structured financial indicators — transaction activity, income and expense signals, banking relationships, cards, merchants and spending categories — usable as credit-scoring features.

Challenge. Banks, mobile operators, lenders, apps and subscription services all format their messages differently, in more than one language. The system had to identify transactions, amounts, currencies, senders and entities across that heterogeneity while confidently discarding the noise — OTPs, marketing, service notices.

SOLUTION

Action. I built a multilingual NLP pipeline on GPT-4o that extracts income, spending behaviour and vendor-category signals from unstructured SMS for thin-file alternative credit scoring. The service is a Python API with transaction filters, entity retrieval, currency processing, aggregators, Docker packaging and unit tests.

The core design masks transaction bodies and matches them against a library of reusable templates. Matched messages yield structured entities immediately; unmatched messages are captured for periodic review so the template library grows as message formats change. I then fine-tuned Llama 3.1 8B with QLoRA — using Unsloth on Colab, trained on synthetic data generated by GPT-4o — to cut inference cost substantially while holding output quality.

Rationale. Template-backed extraction makes recurring transactional messages cheap, traceable and debuggable, and reserves LLM inference for the cases that actually need it. The unmatched-message queue is what keeps the system from silently degrading as banks change their SMS formats. Distilling the work into a small fine-tuned model is what makes per-applicant processing economical at the volumes credit scoring requires.

OUTCOMES

Result. The pipeline gives credit decisioning a structured financial profile for applicants who would otherwise be unscorable, at an inference cost low enough to run per application.

Forsa - LLM-Powered Customer Engagement Assistants

CONTEXT

Situation. Forsa had two customer conversations it could not staff at scale: nudging prospective customers through activation, and reaching borrowers who had fallen behind on payments. Both are high-volume, repetitive, and highly sensitive to tone.

Task. Design assistants that hold real conversations — drawing on customer context and relevant offers, answering service questions, and operating over interactive and WhatsApp channels.

Challenge. This is regulated financial services conducted in Egyptian Arabic. The assistants had to stay grounded in company policy, sound natural rather than machine-translated, carry conversation history across turns, and never produce an unstructured or non-compliant response.

SOLUTION

Action. I built a conversion assistant with GPT-4o that loads customer interests and offer data, selects the relevant offers, and injects customer and background context through the system prompt to steer Egyptian-Arabic conversations toward activation.

Separately, I built a Twilio WhatsApp integration for servicing and collections: it retrieves same-day conversation history, sends template-based conversation starters, and generates JSON-structured follow-up replies via an OpenAI model. Core workflows include contacting overdue applicants, updating customer contact details and encouraging repayment. Company policies and internal knowledge documents are encoded into the system prompt — a deliberately lightweight RAG architecture that keeps responses compliant without standing up a retrieval stack.

Rationale. For a bounded, well-documented policy surface, prompt-grounded context delivers compliance and personalisation without the operational cost of a vector store and retrieval tier. Structured JSON replies keep the model's output inside a schema the application can validate. Twilio provides the channel customers actually read.

OUTCOMES

Result. Two production-shaped assistants covering both ends of the customer lifecycle — activation and collections — automating conversations that previously required an agent for every contact.

INDUSTRIAL & SUPPLY CHAIN OPTIMISATION

Unilever - Real-Time Detergent Manufacturing Recommender

CONTEXT

Situation. A detergent manufacturing tower burns natural gas continuously to dry slurry into powder. Gas is one of the largest controllable costs in the process — but every lever that reduces consumption also risks pushing moisture content or bulk density outside specification, which scraps the batch.

Task. Deliver a decision-support system that runs continuously on live sensor data and recommends machine control settings that cut energy use without compromising product quality.

Challenge. Operators cannot be handed recommendations they must simply trust. The system had to reason over changing slurry flow, nozzle configuration, burner controls, fan speed, tower pressure, multiple temperature readings and recipe-specific constraints — and it had to fail safely, because a quality excursion costs more than the gas it saves.

SOLUTION

Action. I designed and deployed a real-time recommendation system operating on live data from the tower, drawing on 10+ sensors covering temperature at multiple tower levels, moisture content, bulk density and natural-gas consumption. It periodically ingests the current machine state — 25-nozzle water configuration, tower pressure, burner flame strength, slurry feed rate and recipe ID — and recommends nozzle openings, high-pump speed, natural-gas potentiometer setting and burner-fan speed.

The engine uses nearest-neighbour retrieval over historical operating logs: for the current conditions, it finds past machine states that ran under matching conditions with a favourable energy profile, and recommends those settings. I paired this with automated quality safeguards that hand control from the ML recommendation to rule-based corrective actions the moment moisture content or bulk density drifts out of specification — adjusting flame strength, opening additional nozzles or increasing slurry supply — then resume ML-driven recommendations once the process is back in range.

Rationale. Nearest-neighbour retrieval was chosen deliberately over a black-box regressor: every recommendation can be traced back to a real, historical run of the same tower under the same recipe, which is what earns an engineer's trust on a live production line. The rule-based fallback is what makes the whole system safe to deploy — it guarantees that the worst case is a return to conventional operation, not a ruined batch.

OUTCOMES

Result. 23% reduction in monthly natural-gas consumption, with zero production batch failures after the quality safeguards were introduced. The system ran continuously on live manufacturing data as a production decision-support tool.

Unilever - Multi-Variant Manufacturing Scheduler

CONTEXT

Situation. Beyond the tower itself, Unilever had to decide the order and allocation of production for multiple detergent variants against a monthly demand sheet. Planners were building those schedules by hand, reconciling constraints in their heads.

Task. Build a tool that generates production schedules minimising total manufacturing time for a given demand scenario.

Challenge. A valid schedule has to respect line speeds, line efficiencies, variant specifications and base-powder inclusion levels simultaneously. Changing one demand figure ripples through all of them, which is exactly what makes manual scheduling both slow and hard to optimise.

SOLUTION

Action. I built an R Shiny application with reusable scheduling functions, demand-sheet templates, scenario inputs and lookup tables encoding variant specifications, line speeds, line efficiencies and inclusion levels.

Rationale. Parameterising the constraints rather than hard-coding a single schedule turns planning into an experiment: a planner can test a demand scenario against the real manufacturing envelope in seconds, instead of constructing one feasible schedule by hand and hoping it is close to optimal.

OUTCOMES

Result. Planners gained a scenario-testing tool that generates time-minimising schedules directly from a demand sheet while respecting the full constraint set of the manufacturing pipeline.

Procter & Gamble - Supply Network Design Optimisation

CONTEXT

Situation. P&G's distribution network to retail stores had grown incrementally rather than by design — warehouses, hubs, routes and fleet sized by history rather than by optimisation. Nobody could say how much that legacy configuration was costing.

Task. Recommend warehouse, branch and hub locations, delivery routes, fleet requirements and shipment frequencies that minimise total supply-network cost — while supporting brown-field scenarios where existing facilities must stay put.

Challenge. This is four coupled problems at once: geographic allocation, capacity planning, facility location and vehicle routing. Solving them jointly at network scale is intractable, and the answer still has to be something a logistics manager can look at on a map and believe.

SOLUTION

Action. I built a full supply-network-design optimisation platform. It uses k-means clustering to assign stores to warehouses, decomposing the network into serviceable areas, then applies a travelling-salesman algorithm to optimise milk-run delivery routes within each cluster. Inputs are store locations and monthly demand volumes, truck types with capacities and rental rates, working-hours and working-days constraints, and optionally the client's existing network for benchmarking. Outputs are recommended facility placements, supply routes, trip frequencies, fleet requirements, and a cost-comparison matrix spanning every warehouse/hub count combination.

I integrated the HERE Maps API so routing used real road distances rather than straight lines, and built interactive geographic visualisations with Leaflet and an R Markdown interface. A brown-field mode lets selected existing facilities be fixed while the rest of the network is optimised around them.

Rationale. Clustering makes the problem tractable by decomposing it into independent service areas; routing then optimises transport within each. The cost-comparison matrix matters as much as the optimum itself — it shows the client the shape of the cost curve, so they can see what a two-hub network costs versus three and make the trade-off themselves. Brown-field mode is what makes any of it actionable: a recommendation that requires closing every existing facility is a recommendation that never gets implemented.

OUTCOMES

Result. **18% reduction in total supply-network cost** versus the client's existing configuration, delivered with interactive geographic visualisations that let operational stakeholders inspect and challenge each recommendation.

FORECASTING, PRICING & CUSTOMER ANALYTICS

Grinta - Medicine Inventory Replenishment Forecasting

CONTEXT

Situation. Grinta supplies medicines to pharmacies and was losing money at both ends of the same decision: unsold stock accumulating carrying fees, and out-of-stock products turning into missed sales. Both are symptoms of ordering without a forecast.

Task. Forecast replenishment requirements so stock levels are set before supplying pharmacies, not corrected afterwards.

Challenge. The two error directions carry different costs, and pharmaceutical inventory adds shelf-life constraints to an already asymmetric problem — over-ordering is not simply capital tied up, it can be product written off.

SOLUTION

Action. I built a time-series forecasting model for medicine inventory replenishment, predicting how much to stock ahead of supplying pharmacies.

Rationale. Replenishment is a recurring, per-product decision, which makes it exactly the kind of judgement that benefits from being made by a forecast rather than by a person applying a rule of thumb across a long catalogue.

OUTCOMES

Result. Reduced inventory fees on unsold product and avoided missed sales on out-of-stock items — improvement on both sides of the trade-off simultaneously.

Western Union (IBAG) - Branch Cash-Withdrawal Forecasting

CONTEXT

Situation. Western Union branches were being supplied with cash on estimates that were wrong by a wide margin — over-supply error was running at **300%**. Idle cash in a branch is capital that earns nothing and carries security and insurance costs; under-supply turns a customer away at the counter.

Task. Forecast daily cash-withdrawal amounts at individual branch level.

Challenge. Branch-level demand is far noisier than aggregate demand, and the two failure modes are asymmetric — costly on one side, reputationally damaging on the other. The forecast had to be granular enough to drive per-branch supply decisions.

SOLUTION

Action. I developed a time-series forecasting model predicting daily money-withdrawal amounts for each branch.

Rationale. Forecasting per branch rather than centrally is what makes the output actionable, since cash is physically delivered branch by branch.

OUTCOMES

Result. Cash over-supply error reduced from **300% to 20%** — a fifteenfold improvement in supply accuracy.

Wayak Health - Patient Lead Scoring and KPI Analytics

CONTEXT

Situation. Wayak was launching an online drug-delivery subscription and calling patients essentially at random. Roughly 3 in 100 converted, which made outreach economics untenable.

Task. Deliver the exploratory analysis and KPI visibility the business needed to get off the ground, plus a lead-scoring workflow that ranks patients by conversion likelihood.

Challenge. The raw material — patient records, call logs, orders, batch data — needed substantial preprocessing and feature engineering before any repeatable outreach list could be produced, and the output had to fit the sales team's existing weekly rhythm.

SOLUTION

Action. I ran the extensive exploratory analyses the client requested to orient the business, and built a dashboard surfacing their KPIs of interest. On top of that I built the lead-scoring pipeline end to end: data extraction, preprocessing, feature engineering, model training, batch sorting and database injection — with weekly processing of call activity, order history, patient behaviour and disease-related features.

Rationale. The model's output is written back to the database as sorted batches rather than surfaced as scores in a dashboard, because the sales team's workflow is a call list, not an analysis. Making the prediction arrive in the shape the operator already works in is what got it used.

OUTCOMES

Result. Targeting precision improved from 3% to 25% — roughly an eightfold improvement in the conversion rate of outreach.

AMAN Financial Services - Merchant Churn Prediction

CONTEXT

Situation. AMAN held **100GB+ of point-of-sale transaction data covering 100K+ merchants** and had almost no visibility into why merchants stopped transacting. Churn was being discovered after the fact, when the revenue had already gone.

Task. Make the data tractable, deliver insight to stakeholders, and predict which merchants were about to churn.

Challenge. The volume and the transaction-level granularity made preparation the hard part — the modelling could not start until 100GB of raw POS records had been wrangled into merchant-level behavioural features, and the findings had to be communicated to non-technical stakeholders.

SOLUTION

Action. I wrangled the full 100GB+ POS dataset, mined it for insight and delivered those findings to stakeholders through visualisations and reports, then developed an XGBoost model to predict merchant churn.

Rationale. Gradient boosting suits tabular behavioural features derived from transaction aggregates, and gives feature-importance output that made the churn drivers legible to the business — which was half the value of the engagement.

OUTCOMES

Result. Merchant churn prediction at **79% precision**, plus a stakeholder-facing view of the behavioural patterns preceding churn.

London Cab - Driver Positioning and Pricing Simulation

CONTEXT

Situation. Passengers were waiting too long for pickup, and drivers were waiting in the wrong places — supply sitting idle in one part of the city while demand went unserved in another. Separately, the business wanted to change pricing but had no safe way to test what a change would do.

Task. Identify better driver waiting locations, and give the business a way to experiment with pricing strategies before deploying them.

Challenge. Driver supply must be positioned *before* ride requests arrive, which makes this a forecasting problem disguised as a geography problem — and demand hotspots move by hour of day and day of week. Pricing changes, meanwhile, affect both sides of a marketplace at once, so live experimentation is expensive to get wrong.

SOLUTION

Action. I built temporal clustering models to identify optimal driver waiting locations, capturing how demand concentration shifts across time rather than treating the city as a static map. I also designed a dashboard for experimenting with pricing strategies and observing the outcomes through simulation.

Rationale. Clustering on time as well as space is the point: a location that is optimal at 8am is not optimal at 11pm, and a static hotspot map would have positioned drivers correctly only part of the day. Simulation lets the business explore pricing outcomes without risking a live marketplace.

OUTCOMES

Result. Passenger pickup times reduced by 13%, plus a simulation environment for testing pricing strategy before deployment.

Naqla - Truck-Trip Price Analytics and Estimation

CONTEXT

Situation. Naqla's truck-trip prices were set by negotiation and experience rather than by any explicit model, which made quoting inconsistent and left the business unable to explain what actually drove a price.

Task. Analyse trip data to expose the real drivers of price, and build a model that estimates the price of a shipment from its characteristics.

Challenge. Price depends on a wide, interacting set of factors — truck type, shipment weight and goods type, distance travelled, origin and destination, toll fees — none of which is individually decisive.

SOLUTION

Action. I built an analytical dashboard for investigating the patterns governing trip prices across these dimensions, then developed both deep-learning and XGBoost regression models to estimate prices for trips made by trucks of varying size carrying different goods types.

Rationale. The dashboard and the model answer different questions and both were needed: the dashboard tells the commercial team *why* prices vary, which is what they can act on in negotiation, while the model gives a number for a specific shipment. Building both a neural and a gradient-boosted regressor allowed a direct comparison on this feature set rather than an assumption about which family would win.

OUTCOMES

Result. Delivered price transparency the business did not previously have — an analytical view of what drives trip cost, plus automated price estimation for arbitrary shipment configurations.

Sa3ar - Used-Car Fair-Price Estimation

CONTEXT

Situation. Used-car pricing in the local market runs on asking prices and intuition. Neither buyer nor seller has a reference point for what a specific car is actually worth, and no usable dataset existed to build one from.

Task. Assemble the data, then estimate a fair price for a used car from its attributes and condition.

Challenge. The problem started upstream of modelling: the market data existed only as unstructured classified listings, and asking price is a noisy proxy for value.

SOLUTION

Action. I scraped OLX listings to gather car specifications, mileage, tyre condition, accident history and asking price, then built a model to estimate a fair price from those characteristics.

Rationale. Acquiring the dataset was the differentiating part of the work. Once listings across the market are structured, the price surface is learnable — and no one else in the market had that view.

OUTCOMES

Result. A fair-price estimator built on a market dataset assembled from scratch, giving a defensible valuation for a used car from its specifications and condition.

Toyota - Customer Data Quality and Deduplication

CONTEXT

Situation. Toyota's historical record of customer service visits was unreliable. Names carried typos, telephone numbers were implausible or invented, and the same customer appeared as several distinct records — so anything built on this data, from service reminders to lifetime-value analysis, was built on sand.

Task. Cleanse the historical appointment records and identify which visits actually belonged to the same customer.

Challenge. No single field resolved identity. VINs, names, phone numbers and tax numbers were each individually unreliable, and the truth had to be reconstructed from their agreement.

SOLUTION

Action. I cleansed the historical maintenance-appointment data — resolving name typos, removing illogical telephone numbers and similar contamination — then grouped likely duplicate customers by matching across vehicle identification numbers, customer names, telephone numbers and tax numbers together.

Rationale. Entity resolution on any one weak signal produces either missed matches or wrong merges. Requiring corroboration across several independent identifiers is what makes the result trustworthy enough to act on — and a wrong merge in customer data is far more expensive to unwind than a missed one.

OUTCOMES

Result. A cleansed, deduplicated customer history — the prerequisite for any downstream customer analytics, retention modelling or service communication that Toyota wanted to build.

RESEARCH & OPEN SOURCE

dbparser - Open-Source Drug-Database Integration Package

CONTEXT

Situation. Drug-discovery research depends on public databases like DrugBank and KEGG, but every research group was independently writing throwaway code to parse them — solving the same integration problem repeatedly and inconsistently.

Task. Supervise development of an open-source R package that parses and integrates these heterogeneous sources, and set the research direction and feature priorities behind it.

Challenge. The sources differ in schema and format, and the package had to serve genuine drug-discovery workflows rather than merely reading files — while meeting the software-quality bar that open-source adoption demands.

SOLUTION

Action. I co-developed and supervised dbparser, an R package for parsing public drug databases including DrugBank and KEGG. I defined the software requirements and research directions in line with drug-discovery needs, prioritised features, and guided design decisions and comprehensive testing. Parsed heterogeneous sources are structured into a unified R object format — the DrugVerse object — so downstream analysis works against one consistent representation regardless of source.

Rationale. Unifying the output representation is what turns a parser into infrastructure: researchers write analysis once and it works across sources. Centralising recurring integration work into a tested package is the difference between solving the problem once and solving it in every lab separately.

OUTCOMES

Result. **50K+ downloads** and active community adoption — reusable research infrastructure now relied on well beyond the team that built it.

PhD Research - Drug-Target Interaction Prediction

CONTEXT

Situation. Determining whether a drug binds a given protein target experimentally is slow and expensive, so computational prediction is used to prioritise which candidates are worth testing at the bench. The prediction problem is hard for structural reasons: known interactions are sparse, and confirmed positives are vastly outnumbered by unknowns.

Task. Conduct predictive-ML research on drug-target interactions and publish it in peer-reviewed venues.

Challenge. Severe class imbalance, sparse observations, and the need to integrate chemical and biological representations of very different kinds — all under evaluation rigorous enough to survive peer review.

SOLUTION

Action. I developed and evaluated methods spanning ensemble learning, matrix factorisation, manifold learning, dimensionality reduction and class-imbalance-aware learning, including graph-regularised matrix factorisation. The work spanned MATLAB, R, Python, Java and MySQL.

Rationale. These families attack the problem from complementary angles — matrix factorisation recovers latent interaction structure, manifold learning and dimensionality reduction address representation geometry, and imbalance-aware ensembles handle minority-class detection. No single approach addresses all three, which is why the research programme covered them in combination.

OUTCOMES

Result. 4 peer-reviewed papers and 1 book chapter, presented at international conferences, including a **Best Paper Award at InCoB 2016 (BMC Bioinformatics)**. Published in *Briefings in Bioinformatics*, *IEEE/ACM TCBB*, *Methods (Elsevier)* and *BMC Bioinformatics*.

EARLIER ENGINEERING

Earlier Software Projects - BRS and OTMS

CONTEXT

Situation. At United OFOQ I worked on two enterprise systems with very different demands. The Baggage Reconciliation System (BRS, 2008-2010) handles baggage in airport sorting areas, where a mis-sorted bag is a failure a passenger experiences directly. The OFOQ Talent Management System (OTMS, 2010-2011) is a configurable web-based HR platform serving clients with materially different HR processes.

Task. Deliver updates, resolve reported issues, support deployments, and improve product functionality and documentation across both.

Challenge. BRS was operated largely through handheld barcode scanners in a live sorting area — a constrained interface used under time pressure, where usability is an operational requirement rather than a nicety. OTMS had to support client-defined HR processes across payroll, recruitment, health insurance, resource planning and training, with multilingual support and a security model strong enough for confidential employee data.

SOLUTION

Action. On BRS, I added critical updates and new functionality, fixed issues raised internally and by customers, provided direct customer support, improved the visual and operational usability of the scanner-facing interface, updated and extended the documentation for new components, and travelled to Amman, Jordan to carry out on-site deployment.

On OTMS, I worked within the development team, built the reporting functionality using JasperReports, and contributed to data dictionaries, user-defined codes and multilingual support. The stack across both spanned Java EE, JSF/JSP, JavaScript, EJB 3.0, JPA, IBM WebSphere, ActiveVOS, Glassfish, Liferay, IBM DB2 and Oracle.

Rationale. The scanner usability work mattered more than it appears: a UI that costs a handler an extra two seconds per bag is an operational cost multiplied by every bag through the airport. On OTMS, data dictionaries and user-defined codes are the mechanism through which one codebase adapts to each client's HR vocabulary without a fork.

OUTCOMES

Result. Three years of production enterprise software delivery across two live systems — including customer-facing support, documentation ownership and international on-site deployment — the engineering foundation underneath the production ML work that followed.